

Main Idea

We aim to create a Duolingo-inspired programming education platform that transforms how beginners learn to code by eliminating overwhelm through structured, gamified micro-lessons. Unlike fragmented resources, this platform provides a clear, progressive learning path—from core programming fundamentals to advanced topics like OOP, algorithms, and web development—all delivered via multi-format explanations (articles, videos, and guided walkthroughs).

At its core, the platform seamlessly integrates:

1. Interactive Practice & Feedback:

- ▶ An in-browser code editor (supporting Python and JavaScript) with real-time execution, syntax highlighting, and instant error diagnostics.
- ▶ Hands-on exercises featuring model solutions and optional complexity analysis to deepen understanding of code efficiency.

2. Motivational Gamification:

- ▶ XP systems, unlockable levels, daily streaks, and skill badges to reward consistency.
- ▶ Global leaderboards and friendly competitions to fuel engagement.

3. Personalized Progress Tracking:

- ▶ Visual roadmaps highlighting completed, active, and upcoming modules.
- ▶ Mastery dashboards quantifying skill growth and retention.

4. Collaborative Community:

- ▶ Peer code reviews, solution sharing, and discussion forums for collective problem-solving.
- ▶ Optional challenges to apply skills in real-world scenarios.

By combining structured curriculum, instant feedback, and social accountability, the platform turns programming education into an addictive, confidence-building journey—making learning feel like play while ensuring tangible skill development.

Project Scope

1. Comprehensive Curriculum & Content

- ▶ Structured Learning Path:
 - Tiered modules from absolute fundamentals (variables, loops) to advanced domains (OOP, algorithms, web frameworks, databases).
 - Specialized tracks for Python, JavaScript, and full-stack development.
- ▶ Multi-Format Delivery:
 - Concept primers: Short videos + annotated articles.
 - Interactive walkthroughs: Step-by-step coding simulations.
 - Real-world projects: Mini-applications (e.g., weather API integration, todo-list).

2. Intelligent Code Editor

- ▶ Multi-Language Support:
 - Browser-based execution for Python, JavaScript, HTML/CSS, with plans for Java/C++.
- ▶ Enhanced Developer Experience:
 - Real-time syntax + error highlighting, auto-completion, and debugging hints.
 - Code snapshotting: Save/compare solution versions.
- ▶ Accessibility:
 - Dark/light mode, keyboard shortcuts, screen-reader compatibility.

3. Dynamic Interactive Exercises

- ▶ Adaptive Challenges:
 - Exercises auto-adjust difficulty based on user performance.
 - "Fix-the-Bug" tasks: Debug pre-written flawed code.
- ▶ Deep-Dive Analysis:
 - Runtime complexity breakdowns (Big O notation).
 - Memory/performance metrics for optimization practice.
- ▶ Solution Libraries:
 - Model answers + multiple approach comparisons (e.g., iterative vs. recursive).

4. Advanced Gamification System

- ▶ Engagement Mechanics:
 - Daily streaks, skill-specific badges (e.g., "Algorithm Ace"), and XP bonuses for consistency.
 - Unlockable content: Secret lessons or tools for high achievers.
- ▶ Competitive Elements:
 - Global/weekly leaderboards (XP-based).
 - Speed challenges and efficiency contests with peer rankings.

5. Personalized Progress Ecosystem

- ▶ Learning Analytics:
 - Mastery dashboards showing skill proficiency (e.g., "Data Structures: 85%").
 - Time-tracking: Session duration, concepts revisited.
- ▶ Roadmap Customization:
 - Adaptive recommendations for weak areas.
 - Bookmarkable "Playlists" for user-defined goals.

6. Collaborative Community Hub

- ▶ Knowledge Sharing:
 - Solution galleries with upvoting/commenting.
 - Peer review workflows (rubric-guided code critiques).
- ▶ Social Learning:
 - Group challenges: Team-based projects (e.g., build a collaborative app).
 - Q&A forums with mentor verification.
 - Live events: Coding sprints or AMAs with experts.

7. Accessibility & Scalability

- ▶ Mobile-responsive design: Seamless tablet/phone access.
 - ▶ Offline mode: Download lessons/exercises for practice without internet.
 - ▶ API integration: Future compatibility with LMS/CMS platforms.
-

Problem Statement

Learning programming remains a daunting barrier for beginners, exacerbated by *four core gaps* in existing solutions:

1. Structural Deficiency:

- ▶ Resources are fragmented (video tutorials, disjointed exercises) with no coherent progression, leaving learners adrift.
- ▶ *Advanced topics* (OOP, algorithms) feel inaccessible without scaffolded skill-building.

2. Practice-Explanation Misalignment:

- ▶ Passive video/article consumption fails to translate to coding competence.
- ▶ Feedback is delayed or absent, leading to reinforcement of errors and frustration.

3. Motivation Erosion:

- ▶ Isolated learning lacks psychological hooks (rewards, social accountability) to sustain consistency.
- ▶ *80% of beginners quit within 3 months* due to diminishing confidence.

4. Community & Real-World Gap:

- ▶ No safe space to share imperfect code, receive peer reviews, or collaborate.
- ▶ Exercises feel academic, detached from tangible projects or industry practices.

Solution Approach

To bridge these gaps, the platform leverages Duolingo's engagement model fused with developer-centric depth:

A. Structured Yet Adaptive Onboarding

- ▶ Skill Tree Curriculum:
 - *Tiered modules* map concepts from syntax basics → real-world stacks (e.g., Flask/React).
 - Diagnostic quizzes auto-route learners to optimal starting points.
- ▶ Bite-Sized, Multi-Modal Lessons:
 - Concepts taught via < 5-min videos, annotated snippets, and interactive sandboxes—all in one flow.

B. Contextual Practice Engine

- ▶ AI-Assisted Feedback:
 - ▶ Real-time error explanations + debugging hints (e.g., "Your loop exits early: check conditionals!").
- ▶ Exercise Evolution:
 - ▶ Adaptive difficulty: Problems scale complexity based on user mastery.
- ▶ Project Sprints: Build *portfolio-ready micro-apps* (e.g., API-driven weather dashboard).

C. Gamification × Depth

- ▶ Progressive Unlock System:
 - Earn XP/badges for *accuracy*, *efficiency* (e.g., "O(1) Solver"), and *streaks*.
- ▶ Competitive Depth:
 - Leaderboards rank speed (solved in 30s) vs. elegance (least code lines).
- ▶ Complexity Playgrounds:
 - Visualize Big O trade-offs via interactive graph comparisons (e.g., $O(n^2)$ vs. $O(n \log n)$).

D. Community-Powered Growth

- ▶ Collaborative Accountability:
 - Peer review pools: Anonymously critique solutions using *rubric-guided feedback*.
 - Solution Showcases: Compare/upvote multiple approaches (e.g., "Recursive vs. Iterative").
- ▶ Mentor-Verified Support:
 - Expert-vetted discussions in Q&A forums + live coding AMAs.

E. Personalized Reinforcement

- ▶ Predictive Roadmaps:
 - Weakness-targeted challenges (e.g., "Struggling with callbacks? Try these 3 exercises!").
 - ▶ Mastery Analytics:
 - Heatmaps track concept retention + time-to-proficiency across skills.
-

Project Objectives

1. Deliver a Progressive, Mastery-Based Curriculum

- ▶ Modular Skill Tiers: Implement 10+ competency levels (Novice → Architect) with checkpoint assessments for each tier.
- ▶ Cross-Language Tracks: Offer specialized paths for Python (Data Science/Backend), JavaScript (Frontend/Full-Stack), and Algorithms.
- ▶ Real-World Alignment: Integrate industry frameworks (e.g., React, Flask) and tools (Git, APIs) into advanced modules.

2. Build an Intelligent, Adaptive Practice Ecosystem

- ▶ AI-Driven Exercise Engine:
 - Generate personalized problem sets targeting weak areas (e.g., "80% accuracy on recursion? Try these 5 challenges!").
 - Auto-graded projects with rubrics for code quality, efficiency, and creativity.
- ▶ Multi-Layer Feedback:
 - Provide instant syntax corrections, runtime error diagnostics, and performance benchmarks (CPU/memory usage).

3. Gamify Learning with Depth & Nuance

- ▶ Tiered Reward System:
 - Award skill-specific badges (e.g., "Memory Optimizer") + rarity tiers (Bronze → Platinum).
 - "Double-or-Quit" streaks: Bonus XP for consecutive days, reset on skip.
- ▶ Competitive Arenas:
 - Host weekly efficiency leagues (lowest Big O wins) and speed sprints (fastest debugger).

4. Foster Collaborative Expertise

- ▶ Structured Peer Review:
 - Implement rubric-based code critiques (readability, efficiency, edge cases) with upvoted "Top Reviewer" rankings.
- ▶ Mentor-Guided Growth:
 - Verified experts host live "Code Clinics" for Q&A and architecture reviews.
- ▶ Solution Explorer:
 - Curate multiple approaches per problem (e.g., "3 Pythonic Solutions") with complexity comparisons.

5. Enable Hyper-Personalized Tracking

- ▶ Predictive Analytics Dashboard:
 - Visualize skill decay (e.g., "Arrays mastery ↓15% in 2 weeks") and recommend refreshers.
 - Track efficiency gains (e.g., "Reduced solution time by 40% this month").
- ▶ Custom Roadmapping:
 - Let users build goal-oriented playlists ("Prep for FAANG Interviews" → auto-adds relevant exercises).

6. Ensure Accessibility & Scalability

- ▶ Inclusive Design:
 - Support screen readers, keyboard navigation, and color-blind modes.
 - Offer text-to-speech explanations for complex concepts.
- ▶ Infrastructure Goals:
 - Offline-first capability: Download modules + editor for remote learning.
 - API extensibility: Integrate with GitHub/LMS platforms for portfolio syncing.

7. Bridge Theory to Real-World Impact

- ▶ Portfolio Projects:
 - Guide users to build deployable micro-apps (e.g., REST API service, ainteractive dashboard).
- ▶ Industry Challenges:
 - Partner with tech firms for sponsored "real-world" tasks (e.g., "Optimize Shopify's cart algorithm").

8. Drive Community-Led Innovation

- ▶ User-Generated Content:
 - Allow advanced learners to design peer-reviewed exercises (vetted by mentors).
 - ▶ Global Hackathons:
 - Host quarterly themed competitions (e.g., "Sustainable Code Challenge") with expert judging.
-

Project Background

Figma is a versatile, cloud-based design platform widely used for crafting user interfaces, wireframes, and interactive prototypes. It enables designers and stakeholders to collaborate in real time, streamlining the design process from ideation to final output. Figma's rich feature set — including vector editing, component-based design, version history, and a robust plugin ecosystem — makes it central to modern UI/UX workflows. As a browser-based tool, it removes installation barriers and ensures cross-device accessibility.

For more detailed information, you can refer to [Figma](#).

Nuxt.js is a high-level framework built on Vue.js, optimized for developing server-rendered applications and static websites. It offers features like automatic routing, server-side rendering (SSR), static site generation, and a modular architecture. With built-in support for SEO, performance optimizations, and a rich community-driven ecosystem, Nuxt simplifies the development of scalable, high-performance web apps.

For more detailed information, you can refer to [Nuxt.js](#).

.NET is a powerful, open-source development platform created by Microsoft for building modern, scalable, and high-performance applications. It supports multiple languages such as C#, F#, and VB.NET, and enables developers to create applications across web, desktop, mobile, cloud, and IoT environments. Known for its strong type system, robust security features, and extensive class libraries, .NET streamlines development while ensuring reliability and maintainability. With the introduction of .NET Core and now .NET 8, it offers cross-platform support and exceptional performance.

For more detailed information, you can refer to [.NET](#).

Microsoft SQL Server is a robust, enterprise-grade relational database management system (RDBMS) developed by Microsoft. It is designed to store, manage, and retrieve data efficiently while ensuring high performance, security, and reliability. SQL Server supports both structured query language (SQL) for relational data and JSON for semi-structured data, making it suitable for diverse modern applications. It offers advanced features such as ACID compliance, indexing, views, triggers, stored procedures, and built-in analytics through SQL Server Analysis Services (SSAS). With strong support for scalability, data integrity, and transaction management, SQL Server is widely used in enterprise environments for web applications, business intelligence, and large-scale data solutions.

For more detailed information, you can refer to [Microsoft SQL Server](#).

OpenRouter is an open-source API platform that offers a unified interface to multiple large language models (LLMs). It enables developers to seamlessly integrate AI features—such as natural language processing, chat interfaces, and content generation—while managing authentication, fallback strategies, and cost efficiency. OpenRouter simplifies switching between or combining models from different providers.

For more detailed information, you can refer to [OpenRouter](#).

Gemini API is a developer-friendly interface provided by Google to access the capabilities of its Gemini family of large language models (LLMs). It allows developers to integrate advanced AI features into their applications, including natural language understanding, code generation, content summarization, and multi-modal reasoning (text, image, and more). The Gemini API is accessible through Google AI Studio and is designed to support rapid prototyping and scalable deployment of generative AI solutions. With robust security, comprehensive documentation, and seamless integration with Google Cloud, the Gemini API enables powerful, flexible AI experiences across a wide range of use cases.

For more detailed information, you can refer to [Gemini API documentation](#).

Monaco Editor is the highly customizable, in-browser code editor that powers Visual Studio Code. It supports syntax highlighting, IntelliSense, code folding, and more. Lightweight yet powerful, Monaco is perfect for embedding code editing experiences within web applications such as educational platforms, developer tools, or live coding playgrounds.

For more detailed information, you can refer to [Monaco Editor](#).

Cloudflare is a leading web performance and security platform that provides a wide range of services to protect and accelerate websites, APIs, and applications. It acts as a reverse proxy between users and web servers, offering features such as DDoS protection, content delivery network (CDN), SSL/TLS encryption, firewall rules, and performance optimization. By caching content at global edge locations and filtering malicious traffic, Cloudflare helps improve loading speeds, reduce server load, and enhance overall security. It also offers developer tools like Cloudflare Pages and Workers for deploying scalable, serverless applications.

For more detailed information, you can refer to: [CloudFlare](https://www.cloudflare.com/).

Related Work

Elzero Web School

Description:

Elzero Web School is an excellent free resource for Arabic-speaking beginners and intermediate learners who want to build strong web development skills through structured, practical learning.

Techniques Used:

- ▶ External resources and useful tool recommendations included
- ▶ A Q&A section to ask questions and receive community support

Advantages:

- ▶ Step-by-step structured study plans for better learning flow
- ▶ Dedicated learning paths for Frontend, Backend, and Full Stack

Disadvantages:

- ▶ No built-in progress tracking to monitor course completion
- ▶ Users cannot rate or review courses or lessons

Reference: <https://elzero.org>

Codeforces

Description:

A well-known platform that hosts regular contests like Div 1 and Div 2. It includes a robust rating system and editorial support to develop algorithmic thinking.

Techniques Used:

- ▶ Mathematical algorithms
- ▶ Rating system
- ▶ Editorial learning

Advantages:

- ▶ Regular contests with large community participation
- ▶ Detailed editorial explanations
- ▶ Transparent and active rating system

Disadvantages:

- ▶ Interface can be intimidating for beginners
- ▶ Problems often require deep mathematical insight

Reference: <https://codeforces.com>

CodeChef

Description:

An Indian educational platform hosting contests like Long Challenge and Lunchtime, with a vast problem archive and community engagement.

Techniques Used:

- ▶ Long format contests (Long Challenge)
- ▶ Short contests (Lunchtime)
- ▶ Tutorial-based learning

Advantages:

- ▶ Great for long-term learning with multiple contest formats
- ▶ Offers tutorials and mentorship programs

Disadvantages:

- ▶ Sometimes suffers from server lags during contests
- ▶ Problems can be less curated compared to Codeforces or LeetCode

Reference: <https://www.codechef.com>

HackerRank

Description:

Focuses on algorithms, SQL, and data structures with a live coding environment, widely used for tech interviews.

Techniques Used:

- ▶ Structured learning paths
- ▶ Auto-grading system
- ▶ Skill-specific tracks (SQL, AI, etc.)
- ▶ Live coding interface

Advantages:

- ▶ Beginner-friendly interface and structured learning paths
- ▶ Great for practicing specific skills (e.g. SQL, AI)
- ▶ Instant feedback and auto-grading

Disadvantages:

- ▶ Contest competitiveness is relatively low
- ▶ Less challenging for advanced users

Reference: <https://www.hackerrank.com>

LeetCode

Description:

A premier platform for coding interview prep with 2,500+ problems and company-specific questions.

Techniques Used:

- ▶ Interview prep questions
- ▶ Company-tagged problems
- ▶ Weekly contests
- ▶ Solution discussions

Advantages:

- ▶ Focused on technical interview preparation
- ▶ Community solutions and tutorials
- ▶ Weekly contests to benchmark skills

Disadvantages:

- ▶ Some premium features are behind a paywall
- ▶ Less emphasis on advanced algorithms

Reference: <https://leetcode.com>

TopCoder

Description:

One of the oldest platforms, known for SRM (Single Round Matches) and Marathon Matches focusing on complex, long-term problems.

Techniques Used:

- ▶ SRM (Single Round Match)
- ▶ Marathon Match
- ▶ High-difficulty algorithm challenges
- ▶ Real-world modeling problems

Advantages:

- ▶ Highly competitive and professional-grade problems
- ▶ Real-world challenges and big prizes
- ▶ Community of expert coders

Disadvantages:

- ▶ Interface feels outdated
- ▶ Steeper learning curve for newcomers

Reference: <https://www.topcoder.com>

AtCoder

Description:

A Japanese platform offering well-structured contests (ABC, ARC, AGC) with a focus on clean problem statements and difficulty

progression.

Techniques Used:

- ▶ ABC, ARC, AGC contests
- ▶ Clean and structured problems
- ▶ Difficulty progression
- ▶ On-time weekly contests

Advantages:

- ▶ High-quality problems and fair difficulty curve
- ▶ Regular, punctual contests
- ▶ Structured for serious learners

Disadvantages:

- ▶ Japanese-first interface; some translations may be rough
- ▶ Smaller international community than others

Reference: <https://atcoder.jp>

CodinGame

Description:

Gamifies coding challenges with multiplayer and story-based games like Clash of Code and Code vs Zombies.

Techniques Used:

- ▶ Game-based problem solving
- ▶ Real-time multiplayer coding
- ▶ Visual programming challenges
- ▶ Language flexibility (25+)

Advantages:

- ▶ Fun and visual way to learn coding
- ▶ Supports 25+ programming languages
- ▶ Great for casual or team play

Disadvantages:

- ▶ Not focused on algorithm depth
- ▶ Less suitable for serious competitive programming

Reference: <https://www.codinggame.com>

CodeCombat

Description:

An RPG-style platform that teaches Python, JavaScript, and HTML through story-driven games and challenges.

Techniques Used:

- ▶ RPG-style game interface
- ▶ Code-to-play mechanics
- ▶ Curriculum-based learning
- ▶ Beginner visual feedback

Advantages:

- ▶ Ideal for children and beginners
- ▶ Game-based engagement with rewards
- ▶ Offers structured curriculum

Disadvantages:

- ▶ Too basic for experienced developers
- ▶ Some content requires a subscription

Reference: <https://codecombat.com>

Codewars

Description:

Uses "Kata" - short coding exercises - to improve coding progressively with ranking and community feedback.

Techniques Used:

- ▶ Community challenge creation
- ▶ Rank-based progression
- ▶ Peer-reviewed solutions

Advantages:

- ▶ Unique ranking and progression system
- ▶ Community-driven challenges and solutions
- ▶ Good for practicing idiomatic code

Disadvantages:

- ▶ Lacks formal contest system
- ▶ Quality of community challenges can vary

Reference: <https://www.codewars.com>

CheckiO

Description:

Offers gamified learning of Python and JavaScript through short, interactive problem-solving challenges.

Techniques Used:

- ▶ Gamified challenges
- ▶ Code review mechanism
- ▶ Puzzle solving
- ▶ Interactive feedback

Advantages:

- ▶ Fun and visual interface
- ▶ Encourages reviewing others' code
- ▶ Python-focused challenges are especially polished

Disadvantages:

- ▶ Less suitable for advanced algorithm training
- ▶ Limited language support

Reference: <https://checkio.org>

Summary

In this chapter, the tools that will be used in implementation like (Node.js, Nuxt.js, Figma, Gemini API, OpenRouter, Monaco Editor, and CloudFlare). were described. Then the related work was described, (Which is listed in the previous table), and the advantages, disadvantages, and benefits of each one, then compared with the project.

3.1 Feasibility Study

3.1.1 Technical Feasibility

► **Familiarity with Applications:**

- The target learners and educators in our region are already familiar with mobile educational apps and interactive tutorials (e.g. Duolingo, Code.org) in Arabic. Using a gamified, Arabic-language interface makes the platform intuitive; users require no extra training to log in, practice coding, or track progress.
- Core team members have experience building web/mobile learning tools, so we understand typical user workflows (account setup, interactive lessons, community forums).
- Because the UI and content are in Arabic, language barriers are eliminated, further smoothing the learning curve for beginners.

► **Familiarity with Technology:**

- Our team has strong expertise in the chosen tech stack: we have built Vue/Nuxt.js and .NET applications before and are proficient with SQL Server databases. This means we can efficiently develop the front-end UI and the back-end server.
- We have experience with embedding real-time code editors (the platform will use Microsoft's Monaco Editor, the same engine as VS Code) and handling Python and JavaScript code execution on the server.
- We are comfortable working with AI APIs; in past projects we have integrated services (e.g. OpenAI APIs) and we can similarly use OpenRouter to connect to the Gemini API for intelligent hints. Overall, **our familiarity is high** and we have confidence that we have the coding, design, and integration skills needed to implement all planned features.

► **Project Size:**

- The core team will include about 6 members (detailed below), which is appropriate for a mid-size project.
- The platform's scope involves a moderate variety of features: a real-time code editor (supporting Python and JS), gamification mechanics (XP, badges, streaks, daily goals), AI-driven hints, and a community sharing module. This is a mid-level complexity for an experienced team.
- The development timeline is relatively tight: with a target launch by May 2026 (about 6 months from planning start), the schedule is ambitious. However, by assigning parallel sprints for front-end, back-end, and content creation, and by leveraging reusable components (Nuxt/Vue libraries, Monaco Editor, etc.), we believe the team can meet this deadline.

3.1.2 Organizational Feasibility

- ▶ **Champion:** The development team and supervisors provide time and effort for the system.
- ▶ **Project Sponsor:** Dr. Rehab Emad El-Dein
- ▶ **System Users:**
 - ▶ 1. **Learners:** Arabic-speaking students and self-learners who use the platform to learn programming through structured courses, interactive exercises, and real-time code challenges. They earn XP, badges, and streaks as they progress.
 - ▶ 2. **Mentors/Instructors:** More experienced developers or educators who contribute by answering questions, reviewing user code, curating content, and moderating the community. Mentors help ensure quality and provide additional support (similar to Duolingo “language mentors”).
 - ▶ 3. **Administrators/Content Creators:** A small team of admins who upload new course material, monitor the system, and handle technical support.
- ▶ **Project Manager:** Dr. Rehab Emad El-Dein
- ▶ **Development Team Breakdown:** The team is organized into specialized roles with overlapping collaboration to ensure flexibility:

- **Back-end Developers (2):**

Build and maintain the server-side logic in .NET, manage the SQL Server database, and implement APIs. One of the back-end developers will also take on DevOps responsibilities, handling cloud infrastructure, CI/CD pipelines, and security. Together, they will also implement real-time code execution and integration with AI APIs.

- **Front-end Developers (2):**

Develop the user interface using Nuxt.js, ensuring a responsive and intuitive design across desktop and mobile browsers. They will integrate the Monaco editor, implement gamification features (XP, badges, leaderboards), and collaborate closely with the mobile developer to keep design consistent.

- **Mobile Developer (1):**

Focuses on building and optimizing the mobile application version of the platform, ensuring a smooth experience on iOS and Android. Works with front-end and back-end teams for synchronization and performance.

- **AI Engineer (1):**

Specializes in integrating AI features, including intelligent hints and personalized feedback, through APIs like OpenRouter/Gemini. This role also explores adaptive learning models to tailor exercises to each learner's progress.

- **Content & Instructional Design (shared responsibility):**

Instead of dedicated content creators, **all team members will collaborate** on producing and localizing course material in Arabic. This includes designing structured lessons, writing exercises, and embedding gamification mechanics. Team members' technical expertise ensures content is accurate, while shared responsibility distributes workload evenly.

Figure 3.1: Development Team Roles

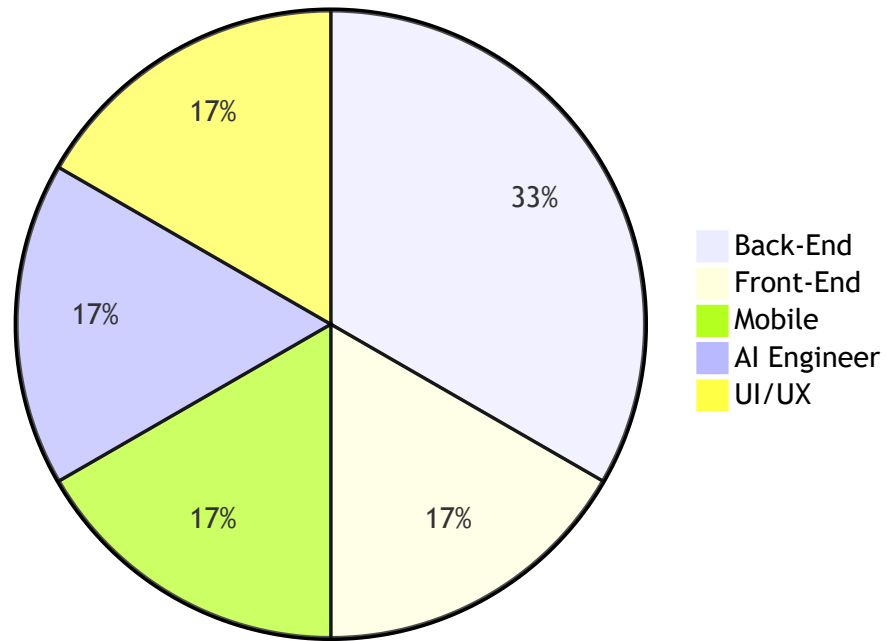


Figure 3.1: Development Team Roles

3.1.3 Economic Feasibility

► **Tangible Benefits:**

- **Course Revenue:** With a pay-per-course model at an average price of \$30 per course, enrolling 5,000 users in Year 1 (our target) would generate roughly \$150,000 in Year-1 sales. As user growth continues, Year 2 and 3 revenues could be, for example, \$225,000 and \$300,000 (assuming 50% year-over-year user growth).
- **Additional Revenue Streams:** We can develop premium content or certification services in later years (e.g. advanced courses, official completion certificates) to create new revenue. Partnerships or bulk licenses with schools or companies could also add income.
- **Economies of Scale:** Because hosting and maintenance costs (see below) are largely fixed per year, each additional user above Year 1 yields mostly profit. For instance, once we cover the \$20,000/year hosting expense and \$50,000/year marketing, further enrollments significantly improve margins.

► **Intangible Benefits:**

- **Learner Engagement:** The gamified, Duolingo-style approach will keep students motivated. Studies show gamification (points, streaks, badges) significantly boosts user engagement and retention. By making coding fun and rewarding, we help learners persist.
- **Education Impact:** Providing coding education in Arabic removes language barriers and makes computer science more accessible. This can broaden participation in tech education and help develop local talent in programming.
- **Community Growth:** Building a collaborative platform fosters a peer-learning community. Learners and mentors share projects and tips, which enhances learning outcomes and provides social value.
- **Brand and Market Position:** Successfully launching this platform will position our team as innovators in Arabic EdTech. Positive reputation and user testimonials will attract future investments, partnerships, and possibly expansion into new topics or markets.

Development and Operational Costs (Years 0–3):

Item	Year 0	Year 1	Year 2	Year 3	Total
Development (one-time)	\$280,000	\$0	\$0	\$0	\$280,000
Dev team salaries	\$180,000	–	–	–	\$180,000
Content creation	\$100,000	–	–	–	\$100,000
Hosting/AI/maint.	\$0	\$20,000	\$20,000	\$20,000	\$60,000
Marketing	\$0	\$50,000	\$50,000	\$50,000	\$150,000
Total Cost	\$280,000	\$70,000	\$70,000	\$70,000	\$490,000

ROI and Cumulative Net:

Metric	Year 0	Year 1	Year 2	Year 3	Total
Revenue	\$0	\$150,000	\$225,000	\$300,000	\$675,000
Total Cost	\$280,000	\$70,000	\$70,000	\$70,000	\$490,000
Net Profit	-\$280,000	\$80,000	\$155,000	\$230,000	\$185,000
Cumulative Net	-\$280,000	-\$200,000	-\$45,000	\$185,000	—

3-Year ROI: $\$185,000 \div \$490,000 \approx 37.8\%$

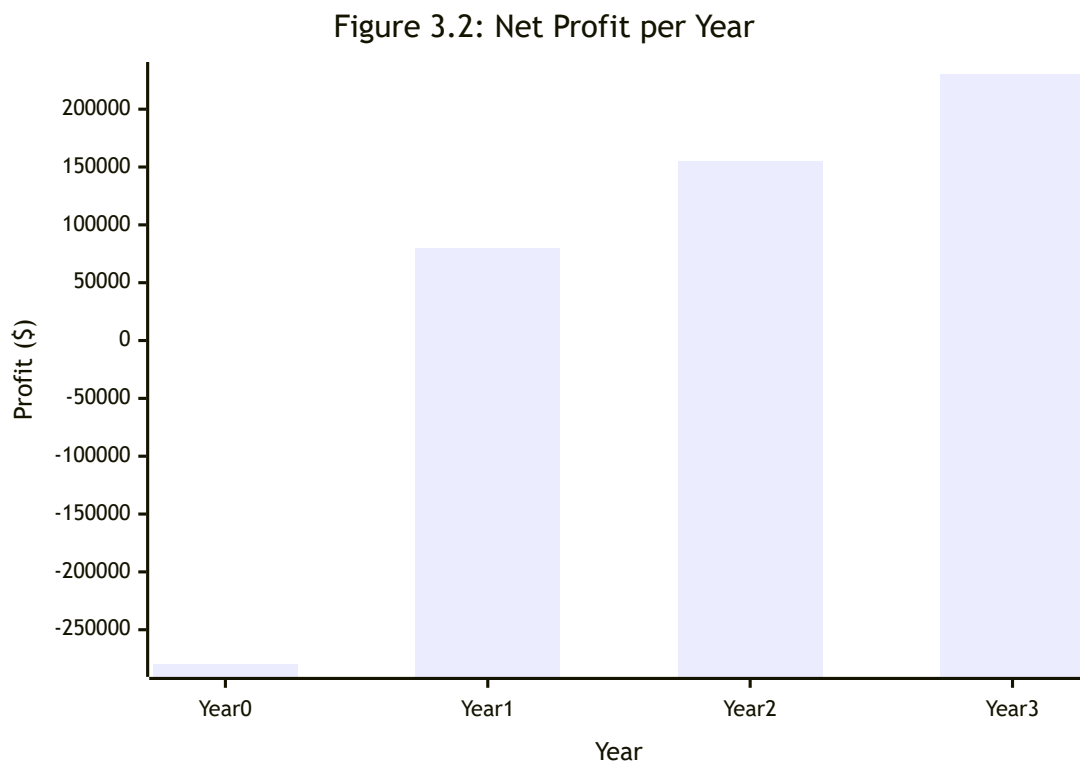


Figure 3.2: Net Profit per Year

Break-even Analysis

The **break-even point** occurs when cumulative revenue surpasses cumulative costs:

- ▶ **Year 1:** Cumulative revenue = \$150,000, cumulative cost = \$350,000 → still negative (-\$200,000).
- ▶ **Year 2:** Cumulative revenue = \$375,000, cumulative cost = \$420,000 → still negative (-\$45,000).
- ▶ **Year 3:** Cumulative revenue = \$675,000, cumulative cost = \$490,000 → **positive cumulative net (\$185,000)** .

Therefore, the project is expected to **break even during Year 3** , after which all additional revenue contributes to profit.

To illustrate:

- ▶ **Break-even revenue threshold:** \$490,000 (total 3-year cost).
- ▶ Achieved at ~ **16,333 paid enrollments** ($490,000 \div \$30$).
- ▶ Based on projected growth, this milestone will be reached in the **third year of operation** .

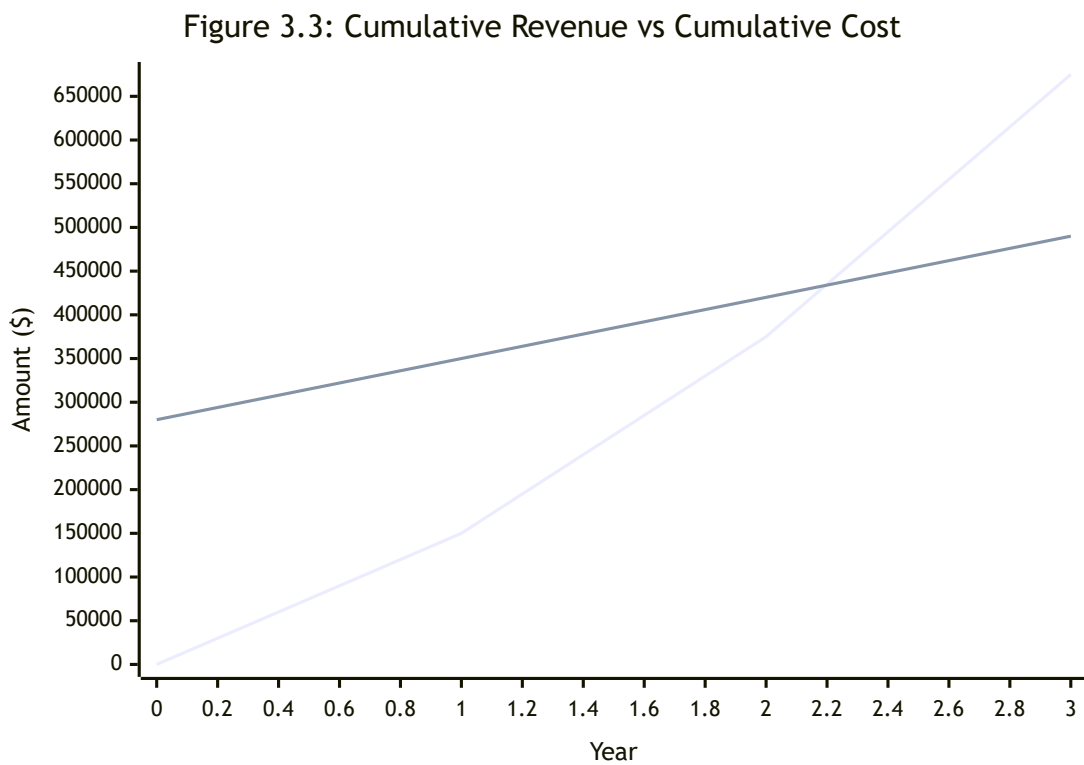


Figure 3.3: Cumulative Revenue vs Cumulative Cost

3.2 Risk Management

This section identifies key risks for the coding education platform across technical, operational, and legal domains. Each risk is assessed with a likelihood (Low/Medium/High), impact (Low/Medium/High), and mitigation strategies. Where appropriate, a qualitative risk matrix is used to emphasize prioritization.

3.2.1 Technical Risks

Risk	Description	Likelihood	Impact	Mitigation
Scalability & Uptime	High traffic or data growth could overwhelm the platform. Without modular architecture and robust testing, performance bottlenecks and downtime can occur.	High	High	Design a scalable, microservices-based architecture; use horizontal scaling (load balancing, CDN, caching); implement automated testing and monitoring to detect and prevent bottlenecks.
External API Integration	Reliance on third-party APIs (e.g. Gemini, OpenRouter) can introduce outages or unpredictable behavior. Third-party services may have downtime or breaking changes.	Medium	High	Vet and monitor external APIs closely (uptime/SLA checks); implement timeouts and retries; use circuit breakers to protect against surges; prepare fallback or degraded modes if an API fails.
Real-time Code Execution	Running user-submitted code in real time is error-prone. Sandbox failures, resource exhaustion, or vulnerabilities could crash the executor, harming reliability.	Medium	High	Isolate execution in secure sandboxes or containers; enforce resource limits (memory/time); continuously test with diverse workloads; scale the execution engine separately; monitor and auto-recover.

3.2.2 Operational Risks

Risk	Description	Likelihood	Impact	Mitigation
Timeline Delays	Requirement changes, scope creep, or underestimation can derail schedules. Over-optimistic estimates may lead to extended deadlines.	High	High	Use thorough upfront planning and clear requirements; apply realistic time estimates with contingency; use agile sprints for incremental delivery and reviews.
Resource Constraints	Limited team size or skill shortages create bottlenecks.	Medium	High	Cross-train staff and onboard talent early; use contingent resources; maintain a pipeline of developers; forecast and reallocate workloads proactively.
Content Development Bottlenecks	Creating high-quality, engaging coding lessons and exercises is time-consuming, which can delay releases or reduce quality.	Medium	Medium	Develop content iteratively with SMEs; reuse or adapt existing materials; employ instructional designers; prioritize high-impact modules first.

3.2.3 Legal & Compliance Risks

Risk	Description	Likelihood	Impact	Mitigation
Data Privacy (Minors)	Collecting data on children raises strict legal requirements (e.g., COPPA, GDPR). Failure to comply can cause severe penalties.	Medium	High	Apply "privacy by design": minimize data collection, encrypt sensitive data, obtain parental consent, maintain clear privacy policies, and conduct regular audits.
Copyright & Licensing	Using third-party or community code/assets risks license infringement. Even one noncompliant license could result in legal or financial penalties.	Low	Medium	Enforce strict review of all content/code; use license scanners; prefer permissive or original content; educate users on plagiarism; remediate infringing material.
Terms-of-Service Violations	Users may post disallowed content (hate speech, copyrighted code, malicious submissions) or cheat, violating the platform's ToS.	Medium	Medium	Publish comprehensive ToS; implement moderation and reporting tools; enforce rules via filters and manual review; respond promptly and revise policies regularly.

3.3 Project plan

TO DO

3.4 Gantt Chart

TO DO

4.1 Function Requirements

4.1.1 User Management System

User Registration and Authentication

- ▶ **FR-1.1:** System shall allow users to create accounts using email, username, and password.
- ▶ **FR-1.2:** System shall support social login (Google, GitHub) for quick registration.
- ▶ **FR-1.3:** System shall implement email verification for new accounts.
- ▶ **FR-1.4:** System shall provide secure password reset functionality via email.
- ▶ **FR-1.5:** System shall enforce strong password policies (minimum 8 characters, special characters).
- ▶ **FR-1.6:** System shall implement two-factor authentication (2FA) as an additional security feature.

Optional Feature: Two-factor authentication (2FA).

User Profiles and Settings

- ▶ **FR-1.7:** System shall allow users to create and edit personal profiles with avatar, bio, and learning goals.
- ▶ **FR-1.8:** System shall allow users to select preferred programming languages (Python, JavaScript, etc.).
- ▶ **FR-1.9:** System shall support multiple language interface options (Arabic, English).

Optional Feature: Language interface options.

User Roles and Permissions

- ▶ **FR-1.10:** System shall support multiple user roles: Learner, Administrator, Content Creator.
 - ▶ **FR-1.11:** System shall implement role-based access control for different platform features.
-

4.1.2 Learning Management System

Course Structure and Navigation

- ▶ **FR-2.1:** System shall organize content into structured learning paths from fundamentals to advanced topics.
- ▶ **FR-2.2:** System shall implement prerequisite-based lesson unlocking mechanisms.
- ▶ **FR-2.3:** System shall provide a visual learning roadmap showing completed, current, and locked modules.
- ▶ **FR-2.4:** System shall support multiple learning tracks (Python, JavaScript, Web Development, OOP, Algorithms).

Optional Feature: Lesson bookmarking.

Lesson Content Delivery

- ▶ **FR-2.5:** System shall support multi-format lesson content (text, videos, interactive demos).
- ▶ **FR-2.6:** System shall provide step-by-step coding walkthroughs with highlighted code segments.
- ▶ **FR-2.7:** System shall include conceptual explanations before practical exercises.
- ▶ **FR-2.8:** System shall support rich text formatting, code syntax highlighting, and embedded media.

Optional Feature: Estimated completion time per lesson.

Adaptive Learning Features

- ▶ **FR-2.9:** System shall track user performance and recommend personalized learning paths.
- ▶ **FR-2.10:** System shall adjust exercise difficulty based on user success rates.
- ▶ **FR-2.11:** System shall provide spaced repetition reminders for previously learned concepts.

4.1.3 Code Editor and Execution Environment

In-Browser Code Editor

- ▶ **FR-3.1:** System shall embed Monaco Editor for in-browser code editing.
- ▶ **FR-3.2:** System shall support syntax highlighting for Python, JavaScript, etc.
- ▶ **FR-3.3:** System shall provide auto-completion and IntelliSense features.
- ▶ **FR-3.4:** System shall include line numbering, code folding, and bracket matching.
- ▶ **FR-3.5:** System shall support multiple editor themes (dark/light mode).
- ▶ **FR-3.6:** System shall provide keyboard shortcuts for common coding operations.

Code Execution and Testing

- ▶ **FR-3.7:** System shall execute user code securely using APIs.
- ▶ **FR-3.8:** System shall display real-time output and error messages.

Optional Feature: Test case validation, execution performance metrics.

Code Analysis and Feedback

- ▶ **FR-3.9:** System shall provide instant syntax error detection and suggestions.
- ▶ **FR-3.10:** System shall offer code optimization suggestions and best practices.
- ▶ **FR-3.11:** System shall compare user solutions with model answers.
- ▶ **FR-3.12:** System shall highlight potential bugs or logical errors using LLM.

Optional Feature: Code complexity analysis with Big O insights.

Blockly Integration

- ▶ **FR-3.13:** System shall provide a block-based workspace with customizable categories (Logic, Math, Control, Events, etc.).
- ▶ **FR-3.14:** System shall allow users to generate source code (JavaScript/Python) from block structures.
- ▶ **FR-3.15:** System shall enable code execution within the application environment.
- ▶ **FR-3.16:** System shall support saving and loading block configurations in XML/JSON format.

Optional Feature: Custom block expansion.

4.1.4 Interactive Exercise System

Exercise Types and Structure

- ▶ **FR-4.1:** System shall support coding challenges, bug fixing, code completion, and multiple-choice exercises.
- ▶ **FR-4.2:** System shall provide clear problem statements with input/output examples.
- ▶ **FR-4.3:** System shall include starter code templates when appropriate.
- ▶ **FR-4.4:** System shall support progressive difficulty within exercise sets.
- ▶ **FR-4.5:** System shall offer optional hints and explanations.

Validation and Grading

- ▶ **FR-4.6:** System shall automatically validate solutions against test cases.
- ▶ **FR-4.7:** System shall track solution attempts and provide retry mechanisms.

Optional Feature: Immediate feedback, partial credit.

Solution Management

- ▶ **FR-4.8:** System shall provide "section of solutions" after successful completion.
- ▶ **FR-4.9:** System shall allow users to submit multiple approaches to the same problem.

Optional Feature: Save and version solutions.

4.1.5 Gamification System

Points and Experience (XP)

- ▶ **FR-5.1:** System shall award XP for completed lessons, exercises, and achievements.
- ▶ **FR-5.2:** System shall implement progressive XP requirements for level advancement.
- ▶ **FR-5.3:** System shall provide XP bonuses for consecutive learning streaks.
- ▶ **FR-5.4:** System shall award bonus points for first attempts.
- ▶ **FR-5.5:** System shall display XP progress bars and level indicators.

Achievement System

- ▶ **FR-5.6:** System shall implement badge categories (completion, mastery, collaboration).
- ▶ **FR-5.7:** System shall provide rare and legendary badges for exceptional achievements.
- ▶ **FR-5.8:** System shall display badge galleries on user profiles.
- ▶ **FR-5.9:** System shall notify users upon earning new achievements.

Optional Feature: Skill-specific badges.

Leaderboards and Competition

- ▶ **FR-5.10:** System shall maintain global and weekly leaderboards.
- ▶ **FR-5.11:** System shall support friend-based leaderboards and challenges.
- ▶ **FR-5.12:** System shall provide leaderboard filtering by time, skill level, or language.

Streaks and Daily Goals

- ▶ **FR-5.13:** System shall track daily learning streaks with visual indicators.
- ▶ **FR-5.14:** System shall send streak reminder notifications.
- ▶ **FR-5.15:** System shall provide streak recovery options.

Optional Feature: Personalized daily goals.

Certification Management

- ▶ **FR-5.16:** System shall display all available certifications with details.
- ▶ **FR-5.17:** System shall allow eligible users to enroll in certification tracks.
- ▶ **FR-5.18:** System shall create a certification dashboard showing progress and deadlines.

4.1.6 Progress Tracking and Analytics

Individual Progress Monitoring

- ▶ **FR-6.1:** System shall maintain comprehensive learning history per user.
- ▶ **FR-6.2:** System shall calculate and display skill mastery percentages.
- ▶ **FR-6.3:** System shall track time spent on topics and exercises.
- ▶ **FR-6.4:** System shall identify strengths and weaknesses.

Performance Analytics

- ▶ **FR-6.5:** System shall track accuracy rates and improvement trends.
- ▶ **FR-6.6:** System shall monitor learning velocity.
- ▶ **FR-6.7:** System shall calculate estimated completion time for coursework.

Visual Progress Representation

- ▶ **FR-6.8:** System shall provide interactive progress roadmaps.
- ▶ **FR-6.9:** System shall display skill trees showing mastered and locked concepts.

Optional Feature: Charts and graphs for analytics.

4.1.7 AI-Powered Features

Intelligent Hints and Assistance

- ▶ **FR-7.1:** System shall integrate AI APIs (Gemini via OpenRouter) for contextual hints.
- ▶ **FR-7.2:** System shall provide progressive hint levels.
- ▶ **FR-7.3:** System shall analyze user code and suggest improvements.
- ▶ **FR-7.4:** System shall generate explanations for complex programming concepts.
- ▶ **FR-7.5:** System shall adapt hint complexity based on user skill.

Personalized Learning Recommendations

- ▶ **FR-7.6:** System shall suggest relevant exercises when users struggle.

Optional Feature: Learning schedules, AI practice problems.

Automated Content Generation

- ▶ **FR-7.7:** System shall generate variations of exercises for extra practice.
 - ▶ **FR-7.8:** System shall generate code examples for abstract concepts.
-

4.1.8 Content Management System

Course and Lesson Administration

- ▶ **FR-8.1:** System shall provide a content creation interface for authorized users.

Optional Feature: Content scheduling, review workflows.

Exercise and Assessment Management

- ▶ **FR-8.2:** System shall allow creation of coding exercises with test cases.
- ▶ **FR-8.3:** System shall support exercise difficulty categorization.
- ▶ **FR-8.4:** System shall validate exercises before publication.

Optional Feature: Exercise cloning and templates.

Multimedia Content Support

- ▶ **FR-8.5:** System shall support video upload or embedding.
- ▶ **FR-8.6:** System shall support content localization for multiple languages.

Optional Feature: Image embedding and interactive content creation.

4.1.9 System Administration

User Management

- ▶ **FR-9.1:** System shall provide an administrative dashboard for managing user accounts.
- ▶ **FR-9.2:** System shall support user role assignment and permission management.

Platform Monitoring and Analytics

- ▶ **FR-9.3:** System shall maintain system logs and audit trails.
- ▶ **FR-9.4:** System shall provide usage statistics and dashboards.

Content Moderation

- ▶ **FR-9.5:** System shall support manual content review and moderation workflows.
- ▶ **FR-9.6:** System shall maintain content removal and user warning systems.

4.1.10 Integration and API Features

Third-Party Integrations

- ▶ **FR-10.1:** System shall integrate with external authentication providers (OAuth or BetterAuth).
- ▶ **FR-10.2:** System shall connect with AI services (OpenRouter, Gemini API).

Platform APIs

- ▶ **FR-10.3:** System shall provide RESTful APIs for web and mobile apps.
- ▶ **FR-10.4:** System shall support webhook integrations.

Mobile and Cross-Platform Support

- ▶ **FR-10.5:** System shall deliver a responsive web interface optimized for mobile.
- ▶ **FR-10.6:** System shall provide mobile-specific features (push notifications).
- ▶ **FR-10.7:** System shall ensure cross-browser compatibility.

Optional Feature: Offline content caching.

4.1.11 Security and Privacy

Data Protection

- ▶ **FR-11.1:** System shall encrypt sensitive user data at rest and in transit.
- ▶ **FR-11.2:** System shall implement secure session management and timeouts.
- ▶ **FR-11.3:** System shall provide privacy controls and data deletion options.
- ▶ **FR-11.4:** System shall comply with GDPR and COPPA standards.

Platform Security

- ▶ **FR-11.5:** System shall implement input validation and sanitization.
- ▶ **FR-11.6:** System shall protect against XSS, CSRF, and SQL injection.
- ▶ **FR-11.7:** System shall secure API endpoints with authentication and authorization.
- ▶ **FR-11.8:** System shall implement rate limiting and DDoS protection.

4.2 Non-Function Requirements

4.2.1 Performance

NFR-1.1: Response Time

- ▶ Landing page load: **2 seconds** (3G or better)
 - ▶ Monaco editor initialization: **3 seconds**
 - ▶ Code execution results: **5 seconds** (standard exercises)
 - ▶ API responses: **1 second** (login, save, submit)
 - ▶ Search/filtering: **2 seconds**
-

NFR-1.2: Throughput

- ▶ Concurrent users: **1,000+** without degradation
 - ▶ Simultaneous code executions: **500+**
 - ▶ Peak submissions: **10,000/hour**
-

NFR-1.3: Resource Utilization

- ▶ Client memory: ≤ **500MB**
 - ▶ Monaco editor: ≤ **200MB**
 - ▶ Database queries: **100ms** (95% of operations)
 - ▶ CDN cache hit rate: **90%+**
-

NFR-1.4: Rendering Performance

- ▶ Interactive elements response: **100ms**
- ▶ Animation frame rate: **60 FPS**
- ▶ Roadmap rendering (100 nodes): **1.5 seconds**

4.2.2 Availability

NFR-2.1: System Uptime

- ▶ Annual uptime: **99.5%** (~43.8 hours downtime)
 - ▶ Maintenance windows: ≤ **4 hours/month** (low-usage periods)
-

NFR-2.2: Service Availability

- ▶ Core features: **24/7** availability
 - ▶ AI hints: **95%** uptime
 - ▶ Leaderboards: **98%** uptime
-

NFR-2.3: Geographic Availability

- ▶ Global accessibility with focus on Arabic-speaking regions
-

NFR-2.4: Graceful Degradation

- ▶ AI unavailable → pre-generated hints available
- ▶ Execution service down → lessons & content accessible

4.2.3 Scalability

NFR-3.1: Horizontal Scalability

- ▶ Stateless architecture for seamless load balancing
 - ▶ Connection pooling for efficient resource management
-

NFR-3.2: User Growth

- ▶ Year 1: **50,000** users
 - ▶ Year 3: **200,000** users (without major changes)
-

NFR-3.3: Content Scalability

- ▶ Launch: **500 lessons** , **5,000 exercises**
 - ▶ Future: **2,000 lessons** , **20,000 exercises**
 - ▶ Support new languages without refactoring
-

NFR-3.4: Database Scalability

- ▶ PostgreSQL with read replicas
 - ▶ Quarterly query plan reviews
-

4.2.4 Reliability

NFR-4.1: Error Rate

- ▶ System error rate: **< 0.5%**
 - ▶ Code execution failures: **< 1%** (system errors only)
-

NFR-4.2: Data Integrity

- ▶ ACID compliance for progress data (Atomicity, Consistency, Isolation, Durability)
 - ▶ Daily backups with 30-day point-in-time recovery
-

NFR-4.3: Fault Tolerance

- ▶ No single point of failure
 - ▶ Database failover: **60 seconds** automatic
-

NFR-4.4: Data Consistency

- ▶ Leaderboards: eventual consistency within **5 minutes**
 - ▶ User profiles: immediate consistency
 - ▶ Cache invalidation: **30 seconds**
-

4.2.5 Interoperability

NFR-5.1: API Standards

- ▶ RESTful design with standard HTTP methods
 - ▶ JSON responses with consistent structure
 - ▶ OpenAPI/Swagger documentation
-

NFR-5.2: Third-Party Integration

- ▶ OAuth 2.0 (Google, GitHub)
 - ▶ Webhook support with JSON payloads
-

NFR-5.3: Data Exchange

- ▶ Export formats: JSON, CSV
 - ▶ Version-control compatible storage
-

NFR-5.4: Browser Compatibility

- ▶ Chrome, Firefox, Safari, Edge (latest versions)
 - ▶ Browsers released within **2 years**
-

NFR-5.5: Mobile Compatibility

- ▶ Responsive: **320px–2560px**
- ▶ Native notifications (FCM/APNs)

4.2.6 Usability

NFR-6.1: Learnability

- ▶ Account setup → first lesson: **5 minutes**
 - ▶ Interactive tutorial: **< 10 minutes**
-

NFR-6.2: UI Design

- ▶ Material Design principles
- ▶ Arabic (RTL) and English (LTR) support
- ▶ 3 font size options

NFR-6.3: Accessibility

- ▶ Keyboard navigation (tab, arrows, enter)
- ▶ ARIA labels and semantic HTML
- ▶ Color-blind modes with alternative indicators

NFR-6.4: Error Handling

- ▶ User-friendly messages (Arabic/English)
- ▶ Contextual help tooltips
- ▶ Searchable help center accessible from all pages

NFR-6.5: Localization

- ▶ Full Arabic/English localization
- ▶ Language switching without progress loss
- ▶ Regional date, time, and number formatting

NFR-6.6: Responsiveness

- ▶ Immediate visual feedback
- ▶ Progress indicators for operations > **1 second**

4.2.7 Maintainability

NFR-7.1: Code Quality

- ▶ Test coverage: **≥ 80%** (critical components)
- ▶ Peer review required before merge

NFR-7.2: Documentation

- ▶ API docs with examples and schemas
 - ▶ Architecture diagrams (system, database, data flow)
 - ▶ Code comments for complex logic
-

NFR-7.3: Modularity

- ▶ Reusable, independently testable components
 - ▶ Versioned migration scripts
-

NFR-7.4: Logging & Monitoring

- ▶ Centralized logs with **90-day** retention
 - ▶ Automated alerts for critical errors
-

NFR-7.5: Deployment

- ▶ Automated CI/CD pipelines
- ▶ Rollback capability: **10 minutes**

4.2.8 Recovery

NFR-8.1: Backup & Restore

- ▶ Daily full + 6-hour incremental backups
 - ▶ 30-day point-in-time recovery
 - ▶ Weekly integrity verification
-

NFR-8.2: Disaster Recovery

- ▶ Semi-annual testing
 - ▶ **RTO** : 4 hours
 - ▶ **RPO** : ≤ 1 hour data loss
-

NFR-8.3: Failure Detection

- ▶ Alert within **2 minutes** of critical failure
-

NFR-8.4: Data Recovery

- ▶ Individual account restoration
 - ▶ Checksum validation with automatic rollback
-

NFR-8.5: Service Recovery

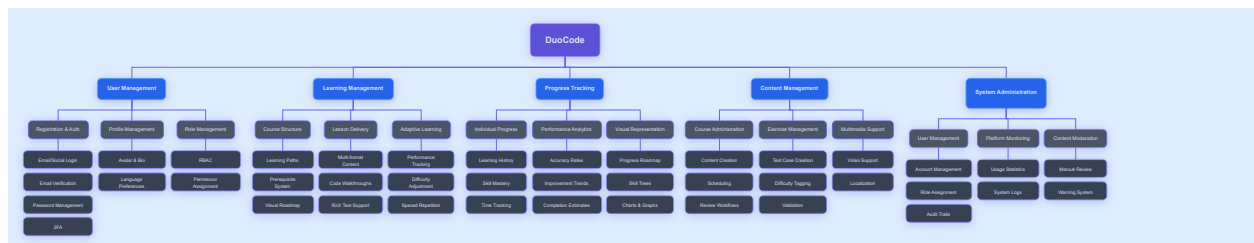
- ▶ Resume after maintenance: **5 minutes**
- ▶ Automatic session restoration
- ▶ Clear incident messaging

NFR-8.6: Transaction Recovery

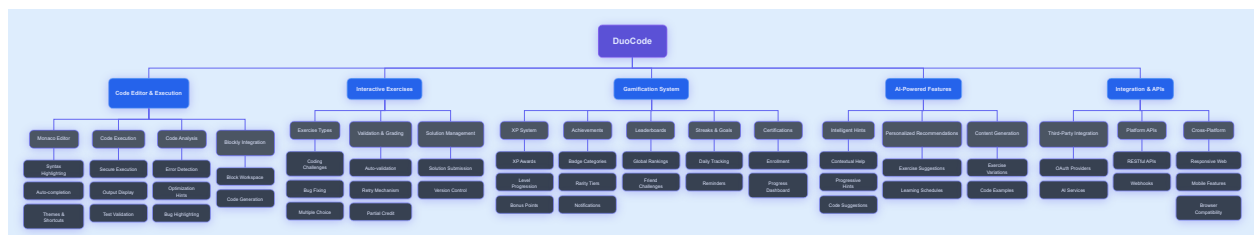
- ▶ Automatic completion or manual review
- ▶ Transaction log maintenance

4.3 Functional Decomposition

Decomposition Diagram - Part 1



Decomposition Diagram - Part 2

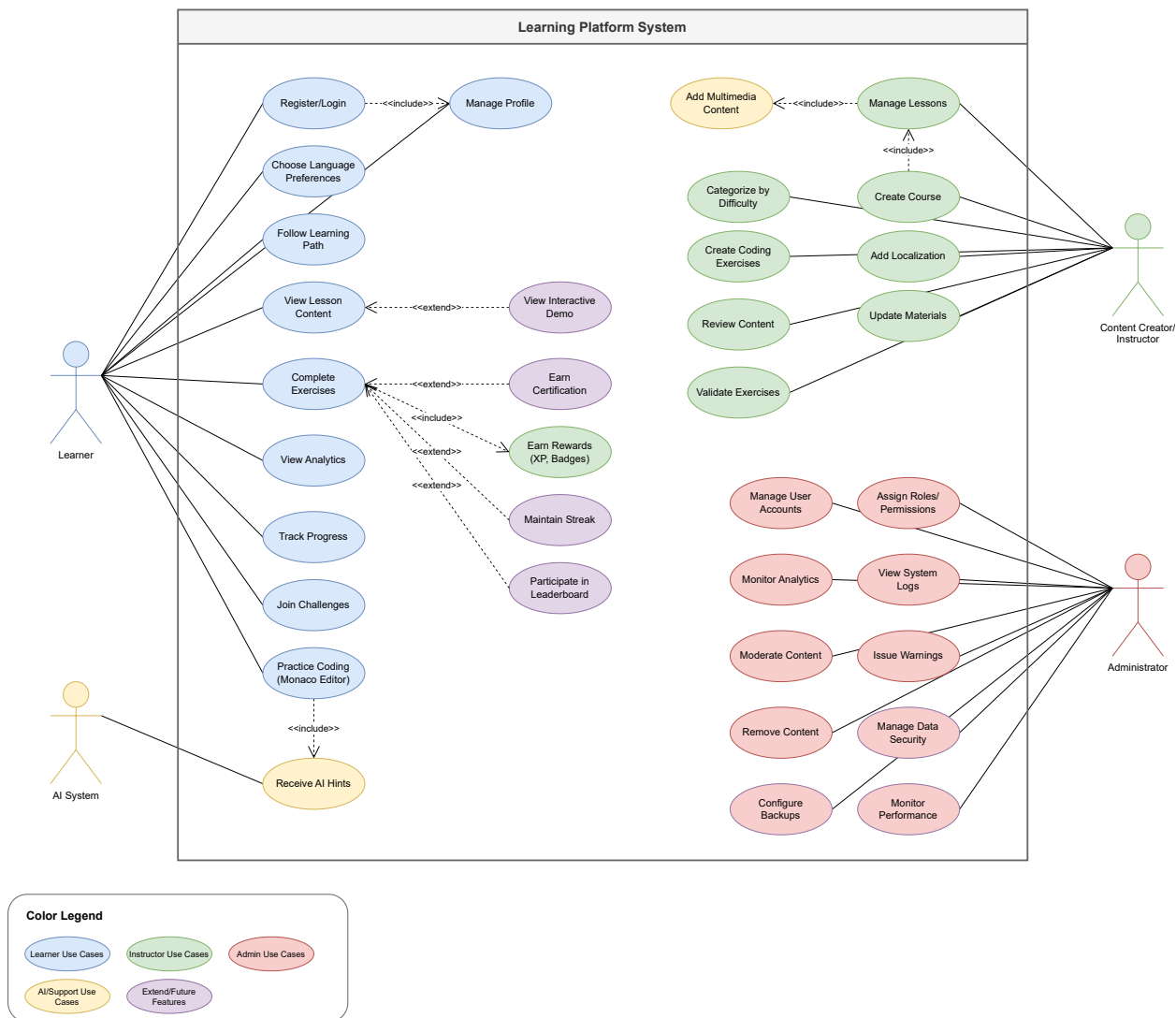


5.1 Use Cases Model

Actor	Goals
Learner	1. Register, log in, and manage their own profiles. 2. Choose preferred programming languages and interface languages. 3. Follow structured learning paths and unlock lessons progressively. 4. Learn via text, videos, and interactive demos. 5. Practice coding using Monaco Editor with syntax highlighting, auto-completion, and real-time feedback. 6. Receive AI-powered hints, explanations, and recommendations. 7. Complete exercises, earn XP, badges, and certifications. 8. Track progress, mastery, and performance analytics. 9. Participate in leaderboards, streaks, and challenges.
Content Creator / Instructor	1. Create and manage courses, lessons, and coding exercises. 2. Categorize exercises by difficulty and validate before publishing. 3. Add multimedia (videos, images) and localized content. 4. Review and update educational materials. 5. Generate content in different formats (text, code, interactive demos).
Administrator	1. Manage user accounts and assign roles/permissions. 2. Monitor platform analytics and logs. 3. Moderate content and enforce warnings or removals. 4. Oversee data security, backups, and system performance. "-" 5. Configure integrations and APIs for external services. "-"
AI System (Gemini / OpenRouter)	1. Provide contextual hints and adaptive learning recommendations. 2. Analyze learner code to detect errors and suggest improvements. 3. Generate additional exercises and code examples. "-" 4. Adjust hint complexity based on user skill level. "-"
System (Platform / App)	1. Securely store and manage user data. 2. Execute and validate user code safely. 3. Deliver responsive and cross-platform experiences. 4. Integrate with external authentication and AI APIs. 5. Enforce data protection, input validation, and DDoS prevention.

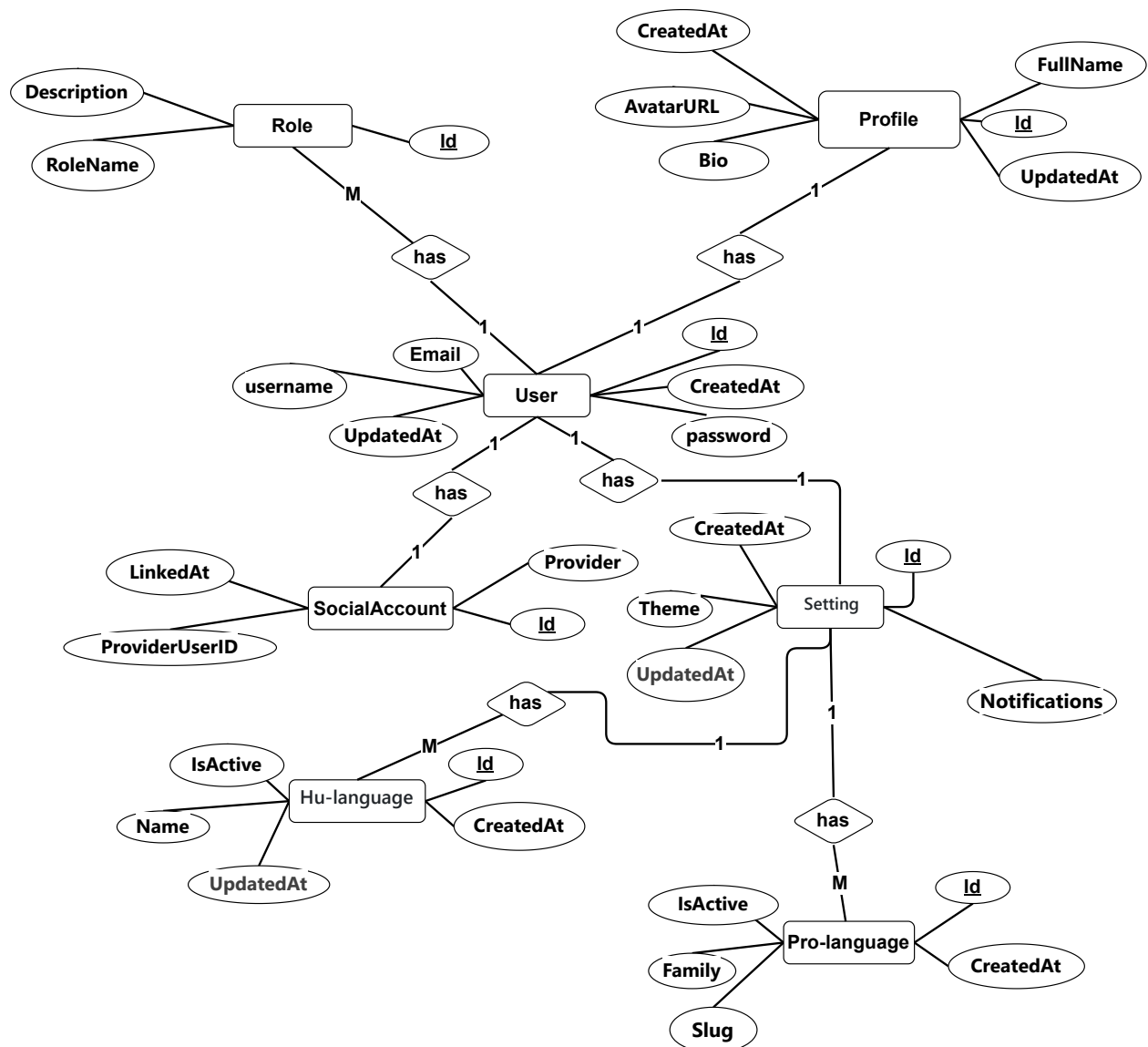
5.1 Use Cases Diagram

Learning Platform System - Use Case Diagram

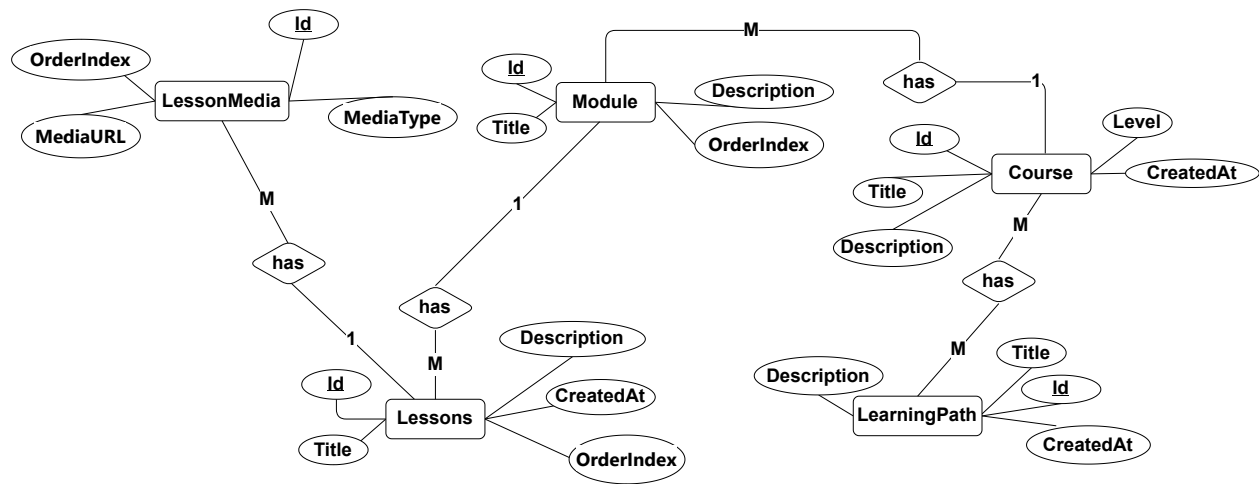


6.1 Entity Relationship Diagram

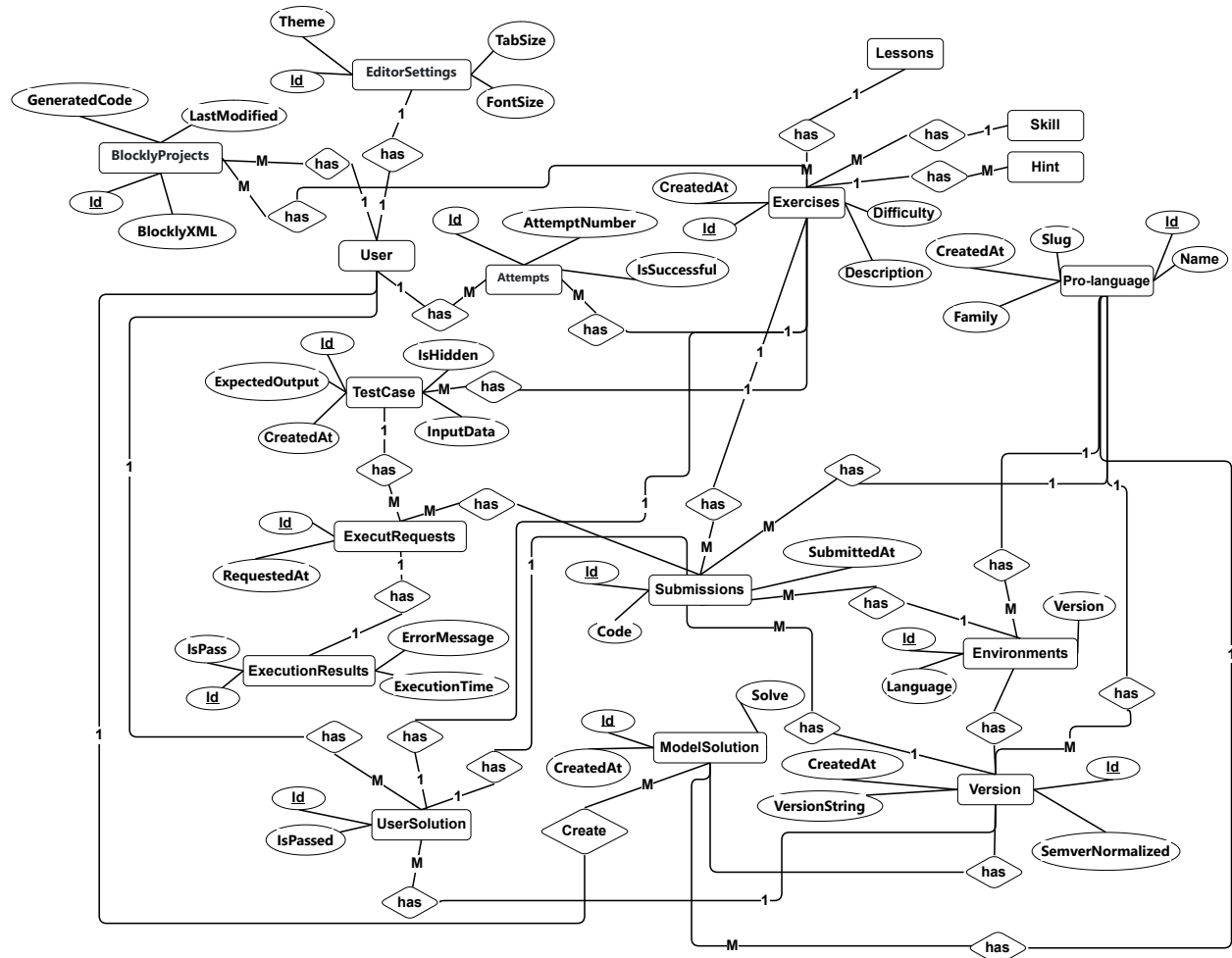
ER Diagram Part 1



ER Diagram Part 2



ER Diagram Part 3



ER Diagram Part 4

